

HOJA DE PRESENTACIÓN

Universidad Autónoma de Zacatecas Unidad Académica de Ingeniería Ingeniería de Software
Materia: Videojuegos Semestre: 8vo semestre

Proyecto Final: MonkeyBlaster

Integrante: Rubén Amauri Cervantes Salmón

Fecha de entrega: 03/06/2026

Repositorio: [RubenAmauri/MonkeyBlaster](#)

INTRODUCCIÓN

El presente reporte documenta el desarrollo de MonkeyBlaster, un videojuego de tipo neon vector shooter desarrollado como proyecto final para la materia de Videojuegos. El juego está inspirado en el clásico Geometry Wars y fue construido íntegramente en Java utilizando el motor jMonkeyEngine 3.8.0.

El proyecto tomó como base una serie de tutoriales publicados en Tuts+ GameDev, originalmente desarrollados para el motor XNA en C#. Dado que el tutorial de jMonkeyEngine estaba incompleto, fue necesario traducir y adaptar varios capítulos directamente desde la versión en XNA, lo cual representó un reto adicional que enriqueció el proceso de aprendizaje.

MonkeyBlaster es un juego de disparos en arena 2D donde el jugador controla una nave espacial, elimina oleadas de enemigos, esquiva agujeros negros con gravedad real, enfrenta un jefe con patrones de ataque, y utiliza power-ups estratégicos para sobrevivir. El juego implementa efectos visuales como un sistema de partículas personalizado, un grid de fondo con simulación física de resortes, y animaciones de sprite. Cuenta con menú principal, menú de pausa, HUD completo con puntuación, vidas y multiplicador, así como un sistema de dificultad progresiva.

El objetivo académico del proyecto fue aplicar los conocimientos adquiridos durante el semestre en transformaciones, texturas, interacción con el usuario, movimiento, física básica y arquitectura de software orientada a componentes dentro de un motor de videojuegos tridimensional adaptado para uso en 2D.

DESARROLLO

1. Arquitectura del proyecto

MonkeyBlaster fue desarrollado en Java utilizando el motor jMonkeyEngine 3.8.0 con el sistema de construcción Apache Ant. La estructura del proyecto sigue las convenciones del SDK de jMonkeyEngine, con el código fuente en `src/mygame/` y los assets en `assets/`.

El juego utiliza el patrón de estados de aplicación (AppState) de jME para separar las distintas pantallas del juego. Cada estado es una clase independiente que maneja su propia lógica, input y recursos:

MenuState gestiona el menú principal con grid animado de fondo, navegación por teclado y música propia. GameState contiene toda la lógica del juego, incluyendo spawning de enemigos, colisiones, partículas, HUD y el sistema del jefe. PauseState maneja la pausa del juego sin destruir el estado de GameState.

El patrón de comportamiento principal es el sistema de Controls de jME, equivalente al patrón Component de otros motores como Unity. Cada entidad del juego tiene uno o más controles que encapsulan su comportamiento. Esta separación de responsabilidades hace el código más limpio y reutilizable.

2. Configuración de la cámara 2D

Dado que jMonkeyEngine es fundamentalmente un motor 3D, fue necesario adaptar la cámara para un juego 2D. Esto se logró con tres configuraciones: proyección paralela para eliminar el efecto de perspectiva, posicionamiento en $Z=0.5$ para que la cámara vea los objetos, y desactivación de la cámara libre FlyByCamera. Todas las entidades del juego se adjuntan al guiNode, donde las unidades de traslación equivalen directamente a píxeles.

3. Carga y representación de entidades

El método `getSpatial()` actúa como una fábrica de entidades visuales. Para cada sprite, crea un nodo padre, carga la textura correspondiente, crea un objeto Picture con las dimensiones exactas de la imagen, desplaza el Picture en $-width/2$ y $-height/2$ para centrar el pivote de rotación, y aplica un material con `BlendMode.AlphaAdditive` que produce el efecto de brillo neon característico al superponer transparencias.

El uso de `inverseMass=0` en las anclas del grid, el uso de `distanceSquared` en las colisiones en lugar de `distance`, y la normalización de vectores de movimiento diagonal son ejemplos del cuidado aplicado en el rendimiento y la corrección matemática del proyecto.

4. Sistema de controles y movimiento

4.1 Jugador

PlayerControl extiende AbstractControl e implementa movimiento en 8 direcciones. Las cuatro teclas de dirección y WASD se combinan en un vector de movimiento que se normaliza cuando hay diagonales activas, garantizando velocidad constante en todas las direcciones. La nave rota hacia la dirección de movimiento. El parámetro tpf (time per frame) se usa en todas las operaciones de movimiento para garantizar independencia del framerate.

4.2 Balas

BulletControl mueve el proyectil en su dirección normalizada, lo rota para alinearse visualmente con la trayectoria, y lo elimina al salir de pantalla disparando una explosión de partículas azules.

5. Sistema de enemigos

5.1 SeekerControl

El Seeker persigue al jugador calculando el vector de dirección entre su posición y la del jugador, acumulando velocidad en esa dirección con una fricción de 0.98f que produce una velocidad máxima natural sin necesidad de limitarla explícitamente. Este es un truco clásico en simulaciones físicas: la velocidad se estabiliza cuando la fricción cancela la aceleración.

5.2 WandererControl

El Wanderer elige una dirección aleatoria inicial en radianes y la ajusta ligeramente cada 6 frames, produciendo un movimiento orgánico y difícil de predecir. Si se acerca a los bordes de pantalla, redirige hacia el centro con una variación aleatoria de hasta 90 grados.

5.3 Sistema de spawn

El sistema de spawn controla la dificultad progresiva mediante la variable inverseSpawnChance, que empieza en 30 y disminuye gradualmente hasta 5, aumentando la frecuencia de aparición de enemigos con el tiempo. Los enemigos siempre aparecen a más de 250px del jugador usando un bucle do-while.

6. Sistema de agujeros negros y gravedad

BlackHoleControl implementa el fade in visual del agujero negro durante su primer segundo de vida, modificando el alpha del material en runtime. Una vez activo, emite partículas púrpuras en un ángulo que rota progresivamente, formando un anillo visual.

El sistema de gravedad en GameState aplica tres modelos físicos distintos según la entidad afectada. El jugador y los enemigos experimentan atracción lineal proporcional a $250/\text{distancia}$. Las balas experimentan repulsión con fuerza negativa. Las partículas usan una función de cuadrado inverso modificada que incluye una componente lateral para producir el efecto de espiral hacia el centro.

7. Sistema de partículas

El sistema de partículas no usa el ParticleEmitter nativo de jME sino sprites individuales con comportamiento propio. Cada partícula es un AbstractControl con velocidad, fricción, escala proporcional a la velocidad, alpha proporcional al tiempo de vida restante, y rebote en los bordes de pantalla.

ParticleManager actúa como fábrica de efectos, ofreciendo explosiones de enemigo en colores HSV aleatorios con 120 partículas, explosión del jugador en blanco y amarillo con 1200 partículas, explosión de bala en azul con 30 partículas, explosión de agujero negro en color basado en tiempo transcurrido con 150 partículas en patrón circular, fuego del escape del jugador con streams central y laterales oscilantes, y spray continuo del agujero negro.

El espacio de color HSV se usa deliberadamente para las explosiones de enemigos porque permite generar colores brillantes y saturados con hue aleatorio, produciendo el efecto neon sin riesgo de generar colores apagados como ocurriría con RGB aleatorio.

8. Grid de fondo

El grid es una simulación física de masas y resortes. En cada intersección existe un PointMass con posición, velocidad y aceleración propia. Las masas del borde tienen $\text{inverseMass}=0$, haciéndolas infinitamente pesadas e inamovibles, lo que ancla el grid en su posición original.

Los resortes implementan la Ley de Hooke con amortiguamiento: $F = \text{stiffness} * \text{deformación} - \text{damping} * \text{velocidad_relativa}$. Solo jalan, nunca empujan. La longitud natural se establece al 95% de la distancia inicial para mantener el grid tenso visualmente.

La integración de Euler simpléctica ($v += a$, luego $p += v$) se usa en lugar de Euler estándar porque conserva energía en sistemas de resortes, evitando que las oscilaciones crezcan indefinidamente.

Las líneas del grid se dibujan mediante Geometry con Mesh de tipo Lines, con controles que actualizan posición, escala y rotación cada frame usando `lookAt()` y `setLocalScale(longitud)`. Las líneas de interpolación adicionales densifican visualmente el grid sin aumentar la cantidad de masas.

9. Sistema del jefe

BossControl implementa un enemigo con 200 puntos de vida, movimiento libre que rebota contra los límites de pantalla y los bordes exteriores del cuadrado de Wanderers, cambio de dirección aleatorio cada 1-3 segundos, animación por sprites individuales ciclando 5 frames cada 150ms, y patrón de disparo semialeatorio de 1 a 3 láseres por grupo con pausas variables.

El jefe aparece al alcanzar múltiplos de 50 puntos. Invoca una pared de 32 Wanderers estáticos formando un cuadrado de 600x600px centrado en pantalla. Genera hasta 3 agujeros negros de forma progresiva conforme pierde vida: el primero al spawnear, el segundo al perder 40% de vida, el tercero al perder 80%.

Los láseres del jefe son proyectiles lentos a 300f que afectan al jugador como los enemigos normales.

10. HUD y sistema de puntuación

La clase Hud gestiona la visualización de vidas, puntuación y multiplicador mediante BitmapText de jME. El multiplicador aumenta con cada kill y expira si el jugador no mata nada en 2 segundos, incentivando el juego agresivo. Cada 2000 puntos el jugador gana una vida adicional. El highscore se persiste en un archivo de texto plano.

11. Sistema de audio

Sound centraliza la gestión de audio con tres categorías de efectos aleatorios (8 explosiones, 4 disparos, 8 spawns) y tres pistas musicales: Space Sprinkles para el menú principal, la música principal para el juego, y Orbital Colossus para las batallas contra el jefe. Toda la música es de Matthew Pablo, obtenida de OpenGameArt.org bajo licencia libre.

12. Dificultades encontradas

Durante el desarrollo se encontraron diversas dificultades técnicas. La ausencia del Capítulo 2 del tutorial de jME obligó a traducir manualmente el código de C#/XNA a Java, siendo necesario comprender las diferencias entre el sistema de iteradores de C# y el sistema de Controls de jME.

El audio en jME requiere que los nodos sean explícitamente no posicionales para audio estéreo, ya que por defecto todos los AudioNode se tratan como fuente de sonido 3D. Esto causó un crash al intentar reproducir música estéreo sin establecer setPositional(false).

La gestión de colisiones en loops que modifican colecciones fue un reto recurrente. Al eliminar entidades durante la iteración fue necesario recorrer los loops en orden inverso y separar la lógica de eliminación de la iteración.

El manejo de spritesheets en jME requiere manipulación manual de buffers UV, lo cual resultó complicado. La solución fue separar los frames del jefe en archivos PNG individuales y cambiar la textura del material en runtime, que es más simple y funciona mejor con la arquitectura de Picture que ya usaba el proyecto.

La incompatibilidad de la clase Rectangle de jME 3.8.0 con el constructor de 4 flotantes que usa el tutorial obligó a modificar el constructor del Grid para recibir ancho y alto directamente.

CONCLUSIÓN

El desarrollo de MonkeyBlaster representó un proceso de aprendizaje intenso y multidisciplinario. Más allá de aprender a usar jMonkeyEngine, el proyecto exigió comprender los fundamentos matemáticos que subyacen a los videojuegos: álgebra vectorial para movimiento y colisiones, física de resortes para el grid, espacio de color HSV para partículas, e integración numérica para simulaciones.

La experiencia de trabajar con un tutorial incompleto y tener que traducir código entre motores y lenguajes diferentes fue desafiante pero muy enriquecedora. Enseñó a no depender de una sola fuente y a desarrollar criterio para adaptar soluciones existentes a contextos diferentes.

El patrón de componentes (Controls en jME) demostró ser una forma elegante de organizar el comportamiento de las entidades del juego. La separación de responsabilidades facilita la depuración, el testing y la extensión del código. Aplicar este patrón de forma consistente desde el inicio del proyecto hizo que agregar nuevas funcionalidades como el jefe fuera relativamente limpio, ya que cada nueva entidad simplemente necesitaba su propio Control.

Un aprendizaje importante fue la diferencia entre un prototipo funcional y un juego completo y pulido. Muchas de las funcionalidades más complejas como el sistema de partículas, el grid físico o el jefe con múltiples fases, requirieron iteraciones múltiples y ajustes de valores que solo se pueden calibrar probando el juego. El balance de dificultad, las velocidades, los tiempos de cooldown, todos esos parámetros involucran tanto criterio técnico como criterio de diseño de juegos.

Personalmente, este proyecto me demostró que el desarrollo de videojuegos es una disciplina que integra programación, matemáticas, diseño visual, diseño de sonido y diseño de experiencia de usuario de forma simultánea. Cada decisión técnica tiene un impacto directo en cómo se siente el juego, y esa conexión entre código y experiencia es lo que hace al desarrollo de videojuegos una disciplina fascinante.

BIBLIOGRAFÍA

Hoffman, M. (2013). Make a Neon Vector Shooter in XNA. Tuts+ GameDev.

<https://dev.tutsplus.com/series/vector-shooter-xna--gamedev-10559>

Tuts+ GameDev. (2013). Make a Neon Vector Shooter in jMonkeyEngine.

<https://code.tutsplus.com/make-a-neon-vector-shooter-in-jmonkeyengine-the-basics--gamedev-11616t>

jMonkeyEngine Documentation. (2025). jMonkeyEngine 3.8.0. <https://jmonkeyengine.org>

Pablo, M. Música: n-Dimensions, Orbital Colossus, Space Sprinkles. OpenGameArt.org.

<https://opengameart.org/users/matthew-pablo>

Zinman-Jeanes, J. Sprites originales de MonkeyBlaster. Tuts+.